MapReduce

Introduction to Data Mining

Seoul National University

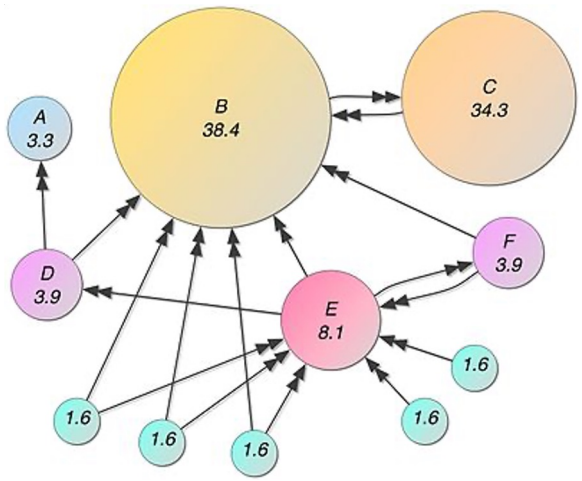
Spring 2026

Outline

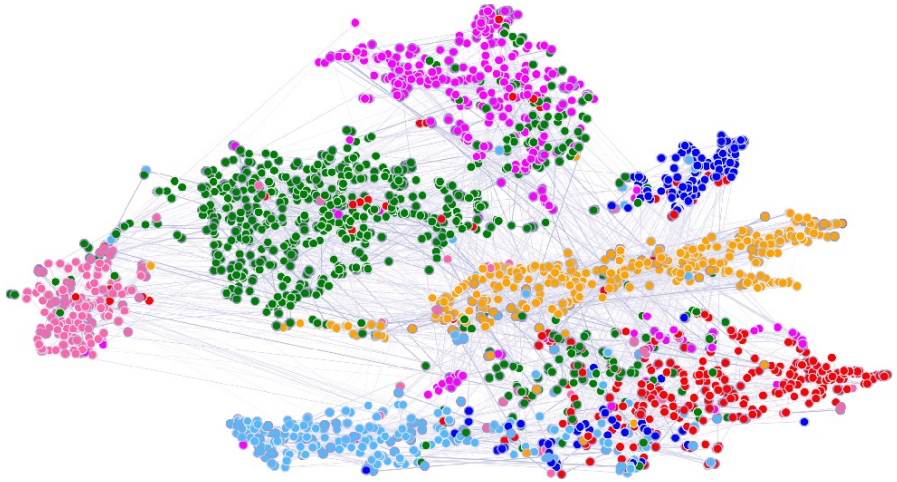
1. **<u>Distributed Computing for Data Mining</u>**
2. MapReduce: Basics
3. MapReduce: Details

Managing Large Amounts of Data

- Ranking Web pages by importance:
  - Iterated matrix-vector multiplication where dimension is many billions.
- Searching friends in a social network:
  - Graphs with hundreds of millions of nodes and many billions of edges.



Source: Spatial



Source: Velickovic et al. (2018)

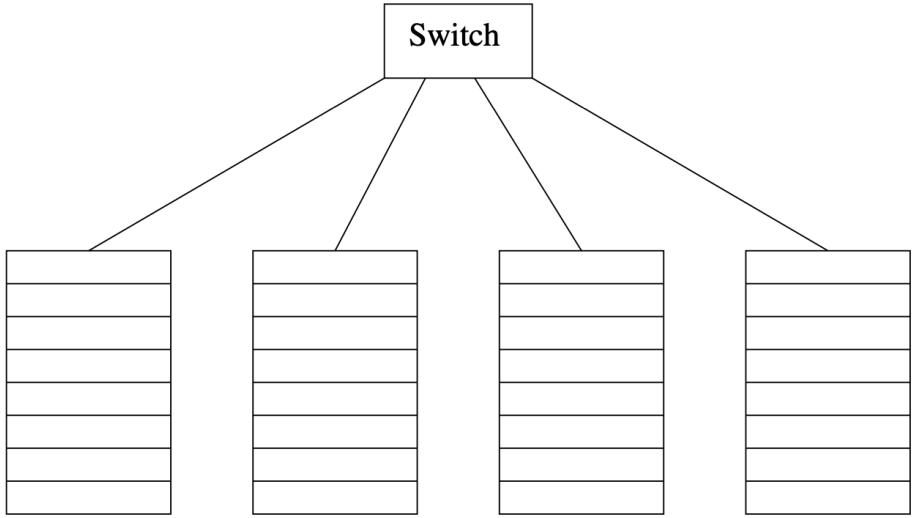
Large-Scale Computing

- Supercomputers are expensive and cannot scale infinitely.
- Instead, we have a **large collection of commodity hardware**.
  - Connected by Ethernet cables or switches.

Source: UMBC

Physical Organization of Nodes

- **Parallel-computing architecture:**
  - **Compute nodes** are stored on racks (perhaps 8 – 64 on a rack).
  - The nodes on a single rack are typically connected by gigabit Ethernet.
  - There can be many racks connected by a switch.



Machine Failures

- **Machines fail frequently:**
  - One server may stay up 3 years (≈ 1,000 days).
  - If you have 1,000 servers, expect to lose 1/day.
  - With 1M machines, 1,000 machines fail every day!
- **Q:** If nodes fail, how to store data persistently?
  - Loss of a single node (e.g., disk crashes)
  - Loss of an entire rack (e.g., network fails)

Storage Infrastructure

- **Distributed File System (DFS)** is a solution.
  - Hadoop Distributed File System (HDFS) or Google File System (GFS).
- **Typical usage pattern:**
  - Huge files (100s of GB to TB).
  - Reads and appends are common.
  - Data is rarely updated in place.

Distributed File System

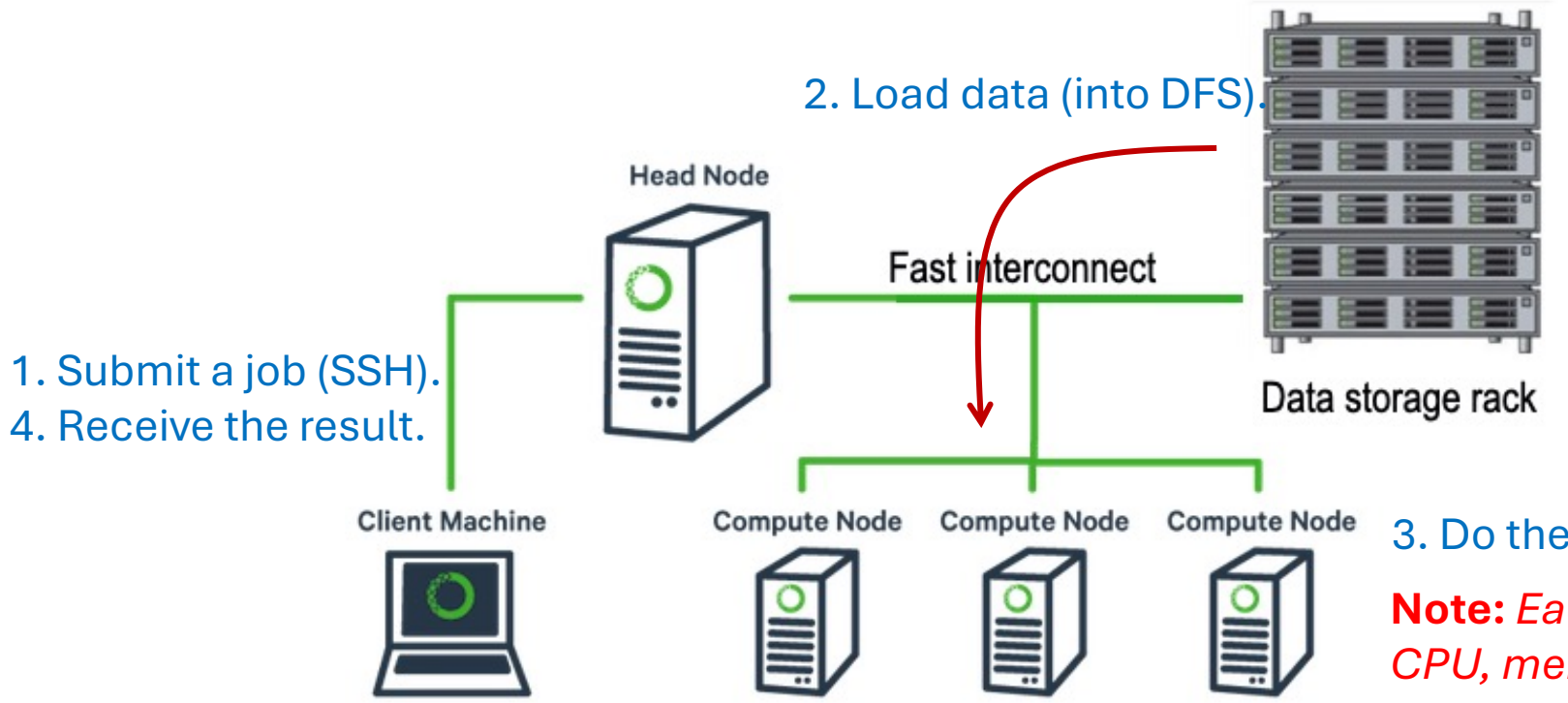
- **Chunk nodes** (or data nodes):
  - Store data divided into chunks, which are typically 16 – 64 MB.
  - Chunks are **replicated** (usually 2x or 3x) at different nodes.
    - Try to keep replicas in different racks.
- **Master nodes** (or head/name nodes):
  - Runs a **master process** that manages the entire file system.
  - Stores the **metadata** (i.e., master file) about where files are stored.
  - Typically has a backup, which takes over when the node fails.

Distributed Computation

- **Q:** How do you distribute computation?
  - Copying large data over a network takes time.
  - We'll learn **MapReduce** and **Spark** as solutions.
- **Idea:** Bring computation to data!
  - Make chunk servers also serve as compute servers.
  - Run a (distributed) algorithm for each chunk.
  - Gather the result, typically smaller than the raw data.

Computing Cluster

- **Q:** How exactly does a distributed cluster work?



3. Do the job.

Note: *Each node has its own CPU, memory, and disk.*

Compute nodes:
Actual computers.
Run workers.
(= chunk servers)
(= compute servers)

Workers:
Virtual processes.
Perform jobs (or tasks).

Source: Corredor and Rodriguez (2021)

Outline

1. Distributed Computing for Data Mining
2. **<u>MapReduce: Basics</u>**
3. MapReduce: Details

MapReduce

- **MapReduce** is a style of programming designed for
  1. Easy parallel programming
  2. Invisible management of hardware and software failures
  3. Easy management of very-large-scale data
- It has several implementations, including Hadoop, Flink, etc.
- The original Google implementation is called “MapReduce”.

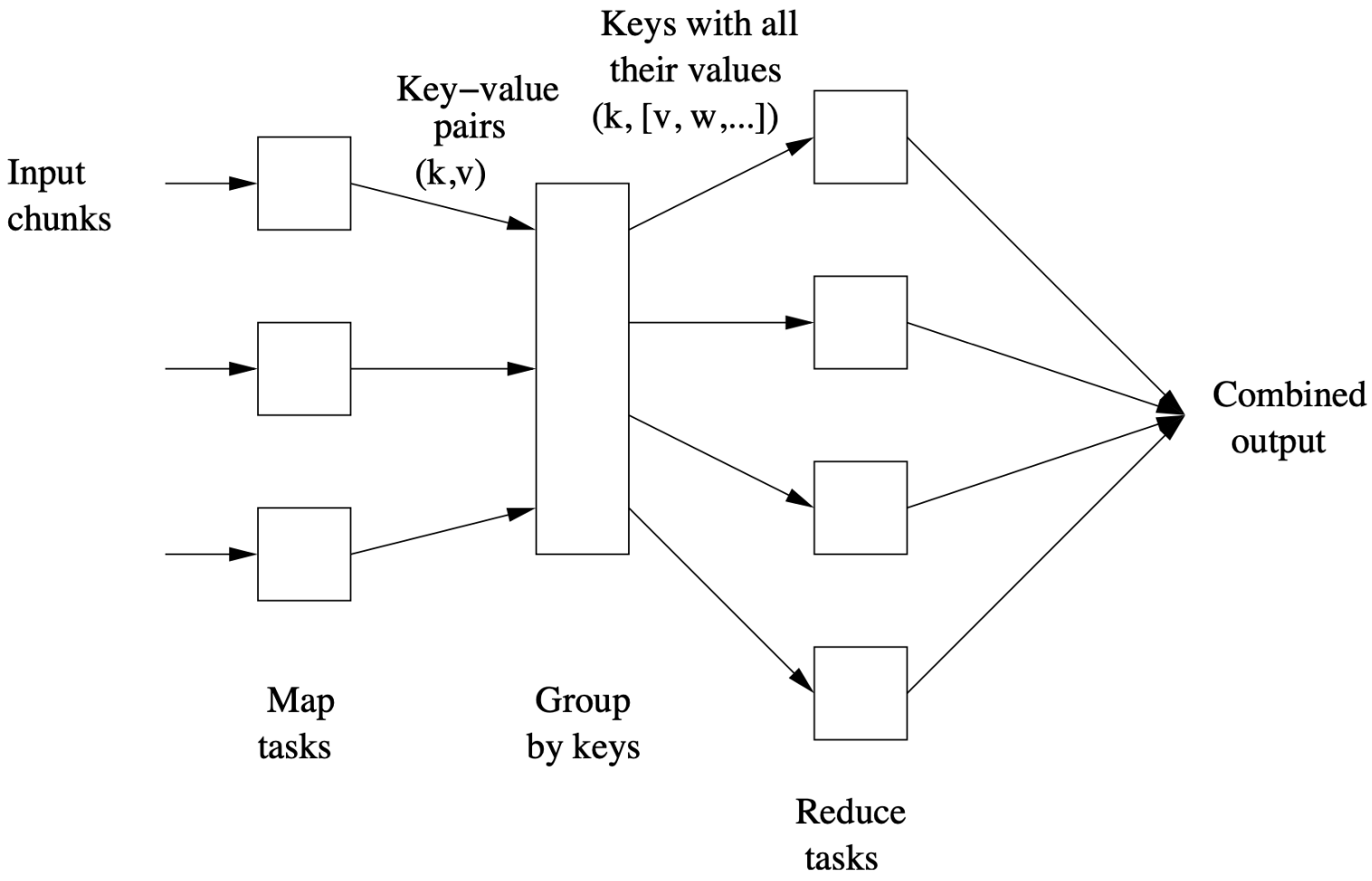
Example: Word Counting

- Suppose we have a huge text document.
- **Goal:** Count the number of times each distinct word appears:
  - E.g., (ball, 30), (basket, 10), (great, 5), and so on.
- **Many applications:**
  - Analyze web server logs to find popular URLs.
  - Analyze market data to find popular items.

MapReduce Components

- **Map:**
  - Apply a user-written *Map function* to each input element.
  - The output of the Map function is a set of key-value pairs.
- **Group by key:**
  - System organizes all the key-value pairs by key.
  - Return key-“value list” pairs.
- **Reduce:**
  - Apply a user-written *Reduce function* to each key-“value list” pair.
  - The output of the Reduce function is a key-value pair.

MapReduce Pattern



Source: Textbook

Map Task

- **Input:**
  - Chunks consisting of elements, which are key-value pairs.
    - Why pairs? To allow composition of several MapReduce processes.
    - Keys of the input (i.e., initial) elements can be ignored.
- **Output:**
  - Zero or more key-value pairs.

Group by Key

- **Input:**
  - Key-value pairs from Map tasks.
- **Output:**
  - The master controller knows the number r of Reduce tasks.
  - Apply a hash function to the keys, producing bucket numbers 0 to $r - 1$.
  - The key-value pairs from each Map task are put into r local files:
    - Each local file is destined for a specific Reduce task.
  - Merge the files from multiple Map tasks and return $(k, [v_1, v_2, \dots, v_n])$.

Reduce Task

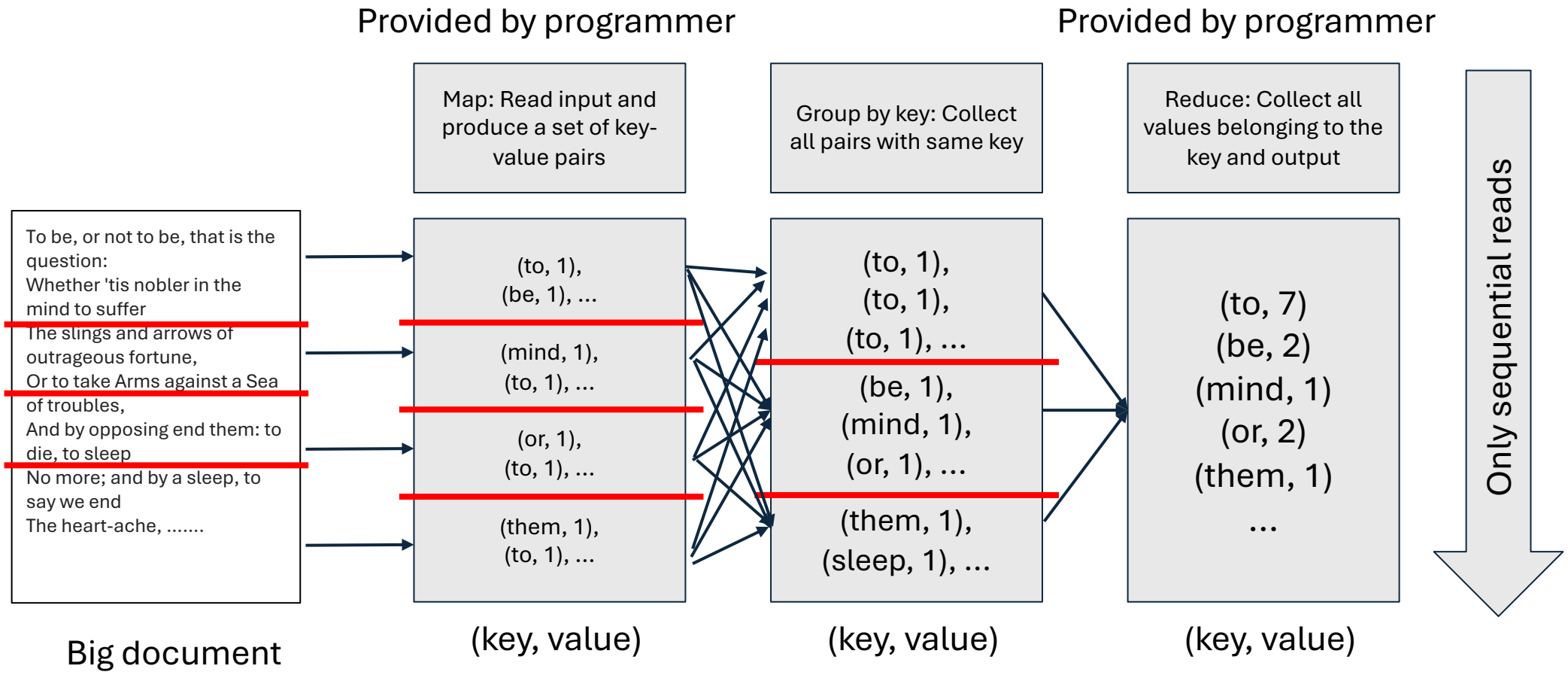
- **Input:**
  - One or more keys and their associated value lists.
    - One Reducer can take several keys.
- **Output:**
  - The Reduce function is applied to each key-“value list” pair.
  - The outputs from all the Reduce tasks are merged into a single file.

MapReduce for Word Counting

```
// key: line index; value: each line of the document
map(key, value):
      for each word w in value:
            emit(w, 1)

// key: a word; values: an iterator over counts
reduce(key, values):
      result = 0
      for each count v in values:
            result += v
      emit(key, result)
```

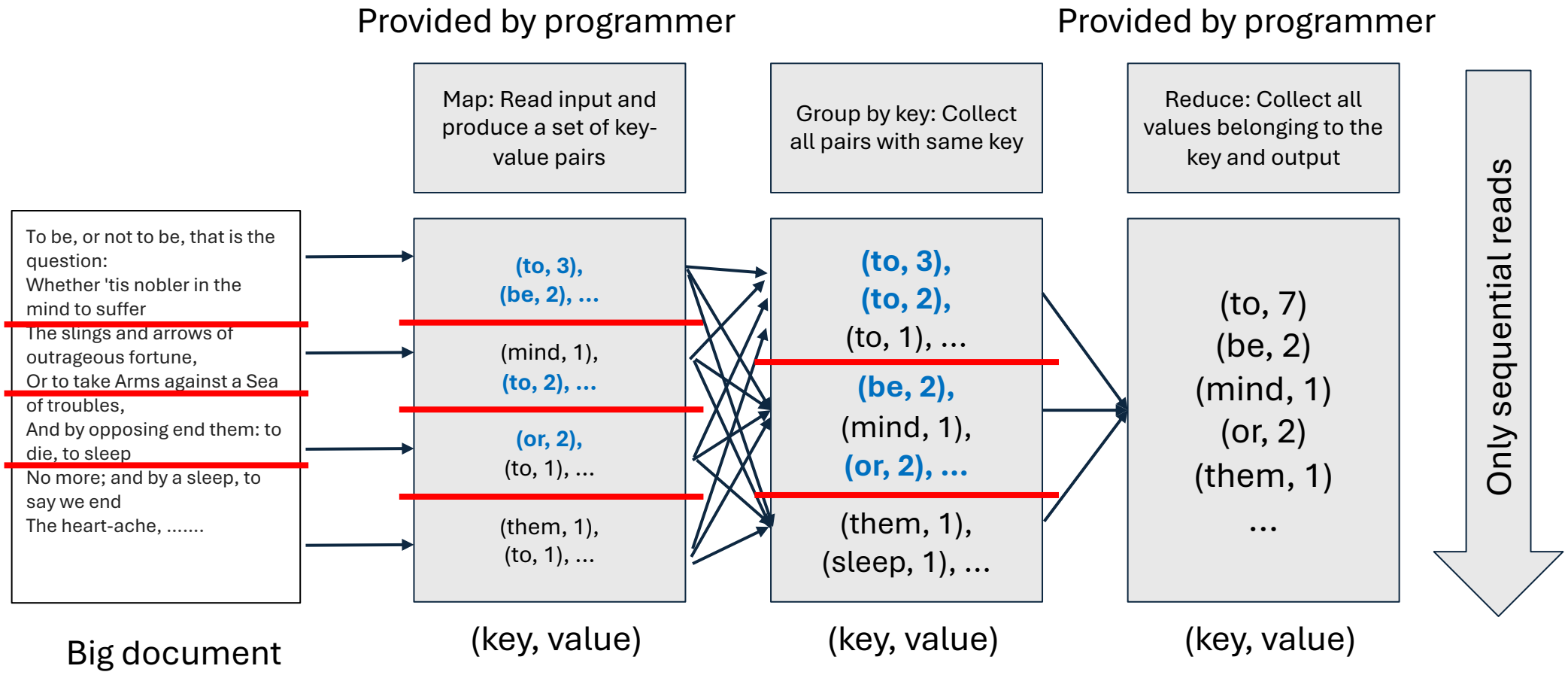
MapReduce for Word Counting



Combiners

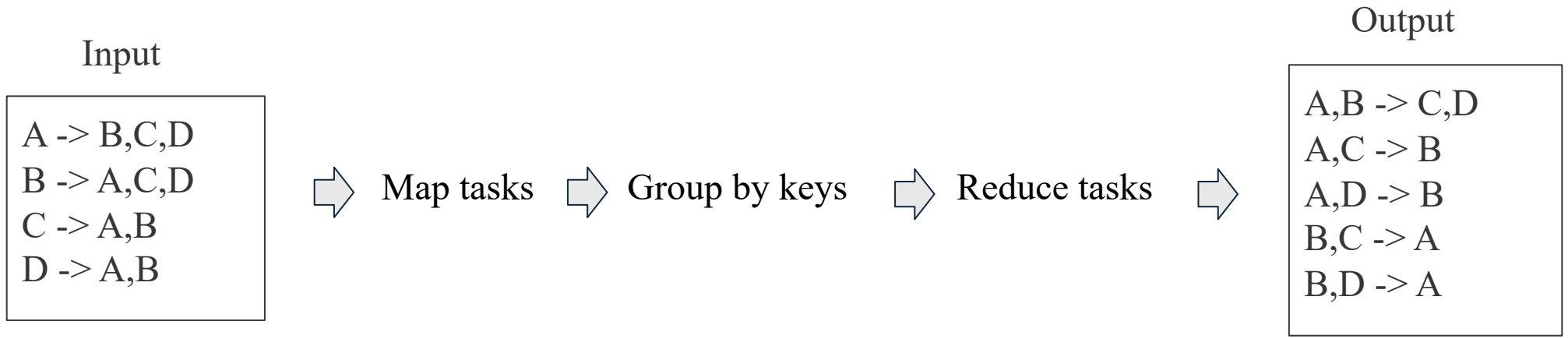
- Suppose the Reduce function is *commutative* and *associative*.
  - **Commutative:** $a + b = b + a$.
  - **Associative:** $(a + b) + c = a + (b + c)$.
  - E.g., word counting, matrix-vector multiplication.
- Then, values can be combined in *any order*.
- **→ The Reducer's work can be pushed to the Map tasks.**
  - E.g., instead of producing $(w, 1)$, $(w, 1)$, …, produce (w, m).
  - The amount of data shuffling for Group By Key is greatly reduced.

Combiner for Word Counting



Pop Quiz

- Let's find common friends of friends on a social network.
  - E.g., A is a friend of B and C at the same time.
- What are the map and reduce functions?

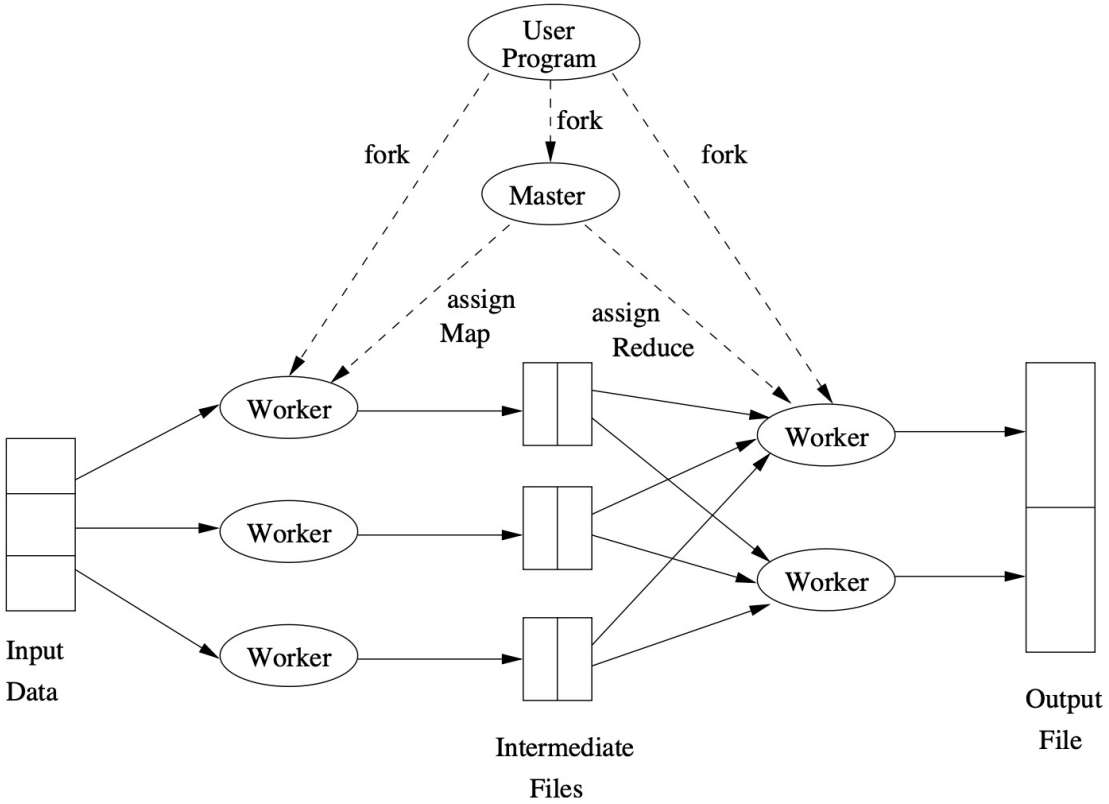


Outline

1. Distributed Computing for Data Mining
2. MapReduce: Basics
3. **<u>MapReduce: Details</u>**

MapReduce Program Execution

- User program forks a **Master controller** and **Worker processes**.
  - A worker handles either Map or Reduce tasks, but not both.



Master Controller

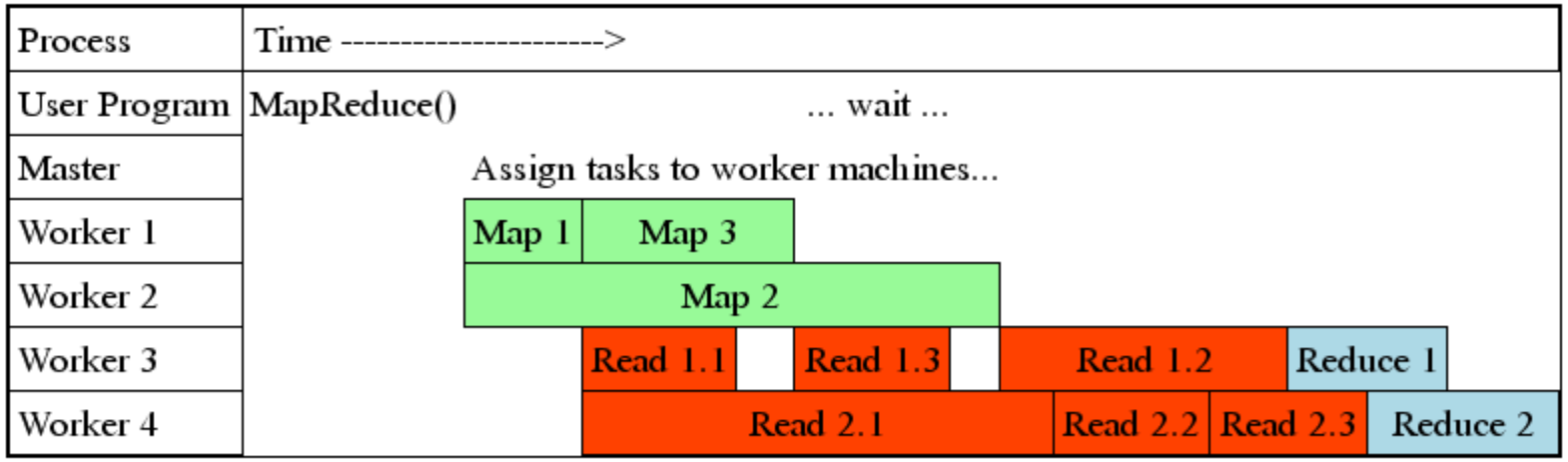
- **Master controller:**
  - Schedules Map and Reduce tasks and assigns them to Workers.
    - One Map task per chunk is reasonable.
    - Reduce tasks should be fewer than Map tasks.
      - Otherwise, too many intermediate files per Map task.
      - Will be discussed later in detail.
  - Keeps track of task status (idle, executing at a Worker, or completed).
  - Worker process reports to Master when it finishes a task.

Map and Reduce Tasks

- **Map task:**
  - Creates a file for each Reduce task on the Map Worker's local disk.
  - The Master is informed of the files and the destination Reduce tasks.
- **Reduce task:**
  - When assigned to a Worker, it is given all the files that form its input.
  - Executes code written by the user and writes its output to a file.

MapReduce Pipelining

- Reduce workers start reading data as soon as a Map task finishes.
- Reduce tasks start after reading all data.



Source: Textbook Slides

Coping with Node Failures

- **Master failure:**
  - Entire MapReduce job must be restarted.
- **Map worker failure:**
  - Detected by Master because it periodically pings Worker processes.
  - All Map tasks assigned this worker have to be redone, even if completed.
    - Output for Reduce tasks reside in compute node and are unavailable.
  - Master informs each Reduce task that input location has changed.
- **Reduce worker failure:**
  - Only in-progress tasks are reset to idle and the Reduce task is restarted.
    - Since the output is stored in DFS, not a local disk.

Number of Reduce Tasks

- Pipelining can be optimized by using one Reduce task per key.
- However, this plan is not usually the best.
  - Overhead associated with each task is created.
  - Often there are far more keys than there are compute nodes.
  - Significant variation in the # of values for different keys.
    - Reduce tasks will exhibit skew in processing time.
- **Rule of thumb:** # of nodes < # of Reduce tasks < # of keys.

Algorithms using MapReduce

- MapReduce is not a solution to every problem.
  - DFS makes sense only when files are large and rarely updated in place.
  - MapReduce is not suitable for frequent database changes.
    - E.g., managing online retails sales (say by Amazon).
- **Original purpose of Google:**
  - To execute large matrix-vector multiplications (for PageRank).
  - Matrix-vector calculations fit nicely into MapReduce.
  - Relational algebra (i.e., SQL) can also use MapReduce effectively.

Matrix-Vector Multiplication

- Suppose we have an $n \times n$ matrix M and a vector $\mathbf{v}$ of length n.
- Matrix-vector product is vector $\mathbf{x}$ of length n whose i-th element is:

$$x_i = \sum_{j=1}^{n} m_{ij} v_j$$

- If $n = 100$, we do not need DFS or MapReduce.
- But in large search engines, e.g., Google, n can be tens of billions.

Matrix-Vector Multiplication

- Assume that v fits in main memory:

Map function
Input: $((i, j), m_{ij})$
Output: $(i, m_{ij}v_j)$

Reduce function
Input: $(i, [mi_1v_1, mi_2v_2, \ldots\ m_{in}v_n])$
Output: $(i, \sum_j m_{ij}v_j)$

M

1	2
1	3

v

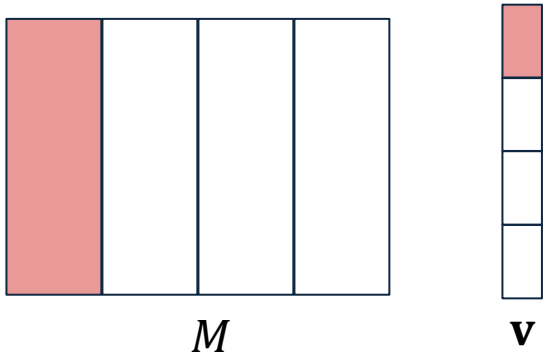
1
2

(1, 1*1 = 1)
(1, 2*2 = 4)
(2, 1*1 = 1)
(2, 3*2 = 6)

(1, [1, 4])
(2, [1, 6])

(1, 5)
(2, 7)

Matrix-Vector Multiplication



- What if the vector $\mathbf{v}$ cannot fit in main memory?
  - Divide M into vertical stripes and divide $\mathbf{v}$ into horizontal stripes.
    - The width of vertical strips = the height of horizontal stripes.
    - Use enough stripes so that one stripe of $\mathbf{v}$ fits in main memory.
  - Each Map task is assigned a chunk from corresponding stripes.
  - The Map and Reduce tasks then run exactly the same as before.

Summary

1. Distributed Computing for Data Mining
   - Distributed File System
2. MapReduce: Basics
   - Map, group by key, reduce, and combiners
   - Example: Word counting
3. MapReduce: Details
   - Coping with node failure
   - Algorithms using MapReduce